

GoGovernment Platform: Comprehensive Project Work Report & Technical Specification

Date: 25 September 2026

Project: `go_government`

Technology Stack: Flutter (Dart), Node.js, Express.js, MongoDB Atlas (Mongoose), Socket.io, Firebase Cloud Messaging (FCM), Hive NoSQL Storage, Android Foreground Services.

Executive Summary

GoGovernment is a civic governance and hyperlocal fair-price commerce platform that bridges citizens, verified government-licensed ration/medical stores, municipal delivery agents (riders), and municipal administrators.

The platform unifies four distinct applications on a shared real-time backend:

1. Citizen App (Mobile): Civic grievance reporting with reward coins, browsing government-approved stores, catalog shopping, cashless wallet/COD checkout, live GPS delivery tracking, and self-service cancellation with automated instant wallet refunds.
 2. Store App (Mobile): Onboarding and license verification, product catalog and inventory management, order processing queue, and automated digital ledger settlements.
 3. Rider App (Mobile): Automated dispatch alerts with 30-second countdowns, turn-by-turn navigation, real-time background GPS tracking via Android Foreground Services, and guaranteed hybrid earnings.
 4. Admin Web Portal (Web): Municipal official dashboard for merchant KYC approvals, grievance monitoring, dispatch tracking, and rider tracking.
-

Architecture Visual Diagram



+-----
---+

2.1 Citizen App Master Flowchart

```
flowchart
    Login["Login & OTP Auth"] --> Home["Citizen Home Dashboard"]

    Home --> StoresSection["1. Verified Stores & Shopping"]
    Home --> GrievanceSection["2. Civic Grievances & Complaints"]
    Home --> BusStopSection["3. Near Bus Stop Locator"]
    Home --> ToiletSection["4. Near Public Toilet Locator"]
    Home --> FeedbackSection["5. Civic Feedback & Survey"]

    1. Stores Flow
    StoresSection --> StoreCatalog["Store Catalog"]
    StoreCatalog --> Cart["Cart with Auto-Store Locking"]
    Cart --> Checkout["Checkout & Delivery Address"]
    Checkout --> PaymentMode["Payment Method"]
    PaymentMode -->|"Wallet Balance"| DeductWallet["Instant Atomic Wallet Debit"]
    PaymentMode -->|"Cash on Delivery"| PlaceCOD["Pending COD Order"]
    DeductWallet --> OrderPlaced["Order Confirmation Screen"]
    PlaceCOD --> OrderPlaced
    OrderPlaced --> LiveTracking["Live Order Status & Rider Map"]
    LiveTracking -->|"Cancel Order"| CancelOrder["Cancellation Type"]
    CancelOrder -->|"If Paid via Wallet"| InstantRefund["Auto MongoDB Refund (+ 1) & Stock Restored"]
    CancelOrder -->|"If Cash on Delivery"| RestoreStock["Stock Restored (No Money Deducted)"]

    2. Grievance Flow
    GrievanceSection --> PostComplaint["Submit Complaint with Geotag & Photo"]
    GrievanceSection --> CoinsWallet["Redeem 100 Coins to 1 Wallet Cash"]

    3. Bus Stop Flow
    BusStopSection --> BusList["Bus Stop List View (Proximity & Routes)"]
    BusList --> BusDirections["Directions Mode (Walk vs Bike ETA)"]
    BusDirections --> BusNav["Turn-by-Turn GPS Navigation"]

    4. Toilet Flow
    ToiletSection --> ToiletList["Public Toilet List (Ratings & Free/Paid)"]
    ToiletList --> ToiletDirections["Directions Mode (Route Polyline)"]
    ToiletDirections --> ToiletNav["Turn-by-Turn GPS Navigation"]

    5. Feedback Flow
    FeedbackSection --> SurveyForm["Community Civic Survey"]
    SurveyForm --> SubmitFeedback["Submit to Backend (/api/feedback)"]
    SubmitFeedback --> AwardCoins["Instant Civic Reward Coins (+50 Coins)"]
    SubmitFeedback --> AdminAlert["Real-Time Socket Alert to Admin Portal"]

    Profile & History
    Home --> Profile["Profile & Orders"]
    Profile --> YourOrders["Your Orders Screen (1-Click Reorder)"]
    Profile --> TxHistory["Transaction History Screen (Refunds & Ledgers)"]
```

2.1.1 E-Commerce & Store Flow

1. Onboarding & Civic Participation:

- Citizens authenticate via Mobile OTP.
- Earn civic incentive reward coins for submitting geotagged civic grievances (potholes, streetlights, sanitation). Coins convert at 100 Coins = 1 Wallet Balance.

• Store Discovery & Cart Building:

- GPS-assisted store finder lists government fair-price ration and medical depots.
- Smart store isolation ensures items from different stores cannot be mixed in the cart.

• Dual-Rail Checkout:

- Wallet Payment: Atomically deducted from the user's live balance.
- Cash on Delivery (COD): Placed without upfront charge; marked `paymentStatus: pending`

• Order Tracking & Cancellation:

- Real-time order progress stepper (`placed` -> `preparing` -> `ready_for_pickup` -> `out_for_delivery` -> `delivered`).
- If cancelled before delivery:
 - Wallet orders: Refunded immediately to `walletBalance`, generating an official `Order Refund` ledger record.
 - COD orders: Cancelled with zero financial overhead; inventory stock is restored.

• Transaction Ledger:

- Dedicated views for Order Refunds, Wallet Top-ups, and Order Purchases.

2.1.2 Near Bus Stop Discovery & Navigation Flow

```
[Near Bus Stop Icon on Home]
  |
  v
{GPS Auto-Detect / Manual Location}
  |
  v
[1. Bus Stop List View]
- List of municipal bus stops sorted by proximity
- Displays Route Numbers (e.g. 500D, 335E, 201R)
- Walking Distance & Average Bus Frequency
  |
```

```

      v
[2. Route & Directions Mode]
- Map overlay with interactive route polyline
- Travel Mode Switcher:
  [0=Ped Walk Mode: 8 mins (650m)] <---> [0=Bike Mode: 2 mins (650m)]
- Estimated Time of Arrival (ETA) via Ola Maps API
      v
[3. Turn-by-Turn Live Navigation]
- Fullscreen interactive FlutterMap navigation view
- Dynamic user position tracking & remaining distance
- 1-Click Compass Re-centering & Exit Navigation

```

Detailed Bus Stop Working Flow

1. Location Resolution: Dynamically reads current GPS coordinates via `Geolocator` or respects user-selected manual locations from `LocationService`.
2. Interactive List: Renders all municipal bus stations within walking and driving radius, complete with routes serviced, shelter badges, and distance.
3. Multi-Modal Routing Engine: Calls `OlaMapsService` to calculate distinct polylines and live ETAs for both pedestrian walking and bike transit.
4. Turn-by-Turn Navigation: Transitions state machine from `list` -> `directions` -> `navigation`, giving citizens a live turn-by-turn guidance display.

2.1.3 Near Public Toilet Facility Locator Flow

```

[Near Toilet Icon on Home]
  |
  v
{Municipal Sanitation Database Query}
  |
  v
[1. Toilet Facilities List Screen]
- Facilities ordered by nearest distance (e.g. 180m, 450m, 1.2km)
  |
  v
[2. Route Preview & Directions Screen]
- Interactive map showing current user pin to public facility
- Route options: Pedestrian Walkway vs Road Drive
- Live duration estimates via Ola Maps Routing API
  |
  v
[3. Live Turn-by-Turn Guidance]
- Real-time tracking to exact facility entrance

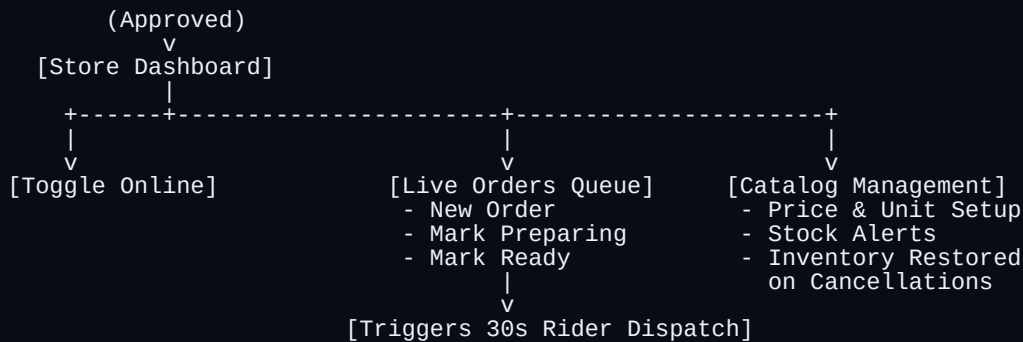
```

2.1.4 Civic Feedback & Community Survey Module

```

[Civic Feedback & Survey Screen]
  |
  v
[4-Pillar Civic Infrastructure Survey]
+-----+

```

Detailed Store App Working Flow

1. Onboarding & KYC: Merchant submits registration with business license number, address, bank account details, and storefront image.
2. Order Lifecycle Execution:
 - Receives instant audio-visual order alerts via WebSocket room `store:<storeId>`.
 - Transitions order status: `placed` -> `preparing` -> `ready_for_pickup`.
 - Marking `ready_for_pickup` automatically initiates the 30-second rider dispatch engine.
- Automated Settlement:
 - On delivery completion, the store receives direct ledger credit:
`grandTotal - deliveryCharge`.

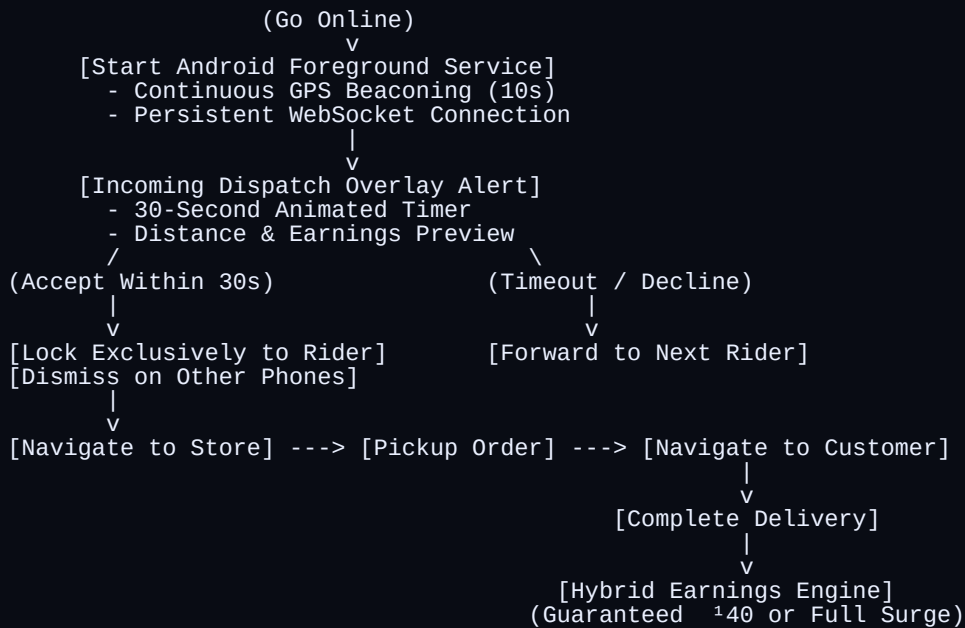
2.3 Rider App Screen Flowchart

```

flowchart TD
    RiderLogin["Rider Login & Profile"] --> DutyToggle["Duty Status Toggle"]
    DutyToggle --> Offline["Offline"]
    Offline --> IdleState["Rider Inactive / Standby"]
    DutyToggle --> GoOnline["Go Online"]
    GoOnline --> StartFgService["Start Android Foreground Service"]
    StartFgService --> GPSBeacon["Background GPS Ping every 10s"]
    GPSBeacon --> ListenDispatch["Listen on Socket Room 'riders'"]
    ListenDispatch --> OrderAlert["Incoming Dispatch Alert with 30s Timer"]
    OrderAlert --> Choice["Rider Decision"]
    Choice --> Timeout["Timeout / Reject"]
    Timeout --> DismissAlert["Dismiss & Forward to Next Rider"]
    Choice --> AcceptOrder["Accept Order"]
    AcceptOrder --> LockOrder["Lock Order Exclusively to Rider"]
    LockOrder --> CancelTimer["Server Cancels Alert on Other Devices"]
    LockOrder --> NavStore["GPS Route Navigation to Store"]
    NavStore --> PickupDone["Pickup Items from Merchant"]
    PickupDone --> NavCustomer["GPS Route Navigation to Citizen"]
    NavCustomer --> CompleteDelivery["Deliver Items & Collect Cash if COD"]
    CompleteDelivery --> PayoutEngine["Hybrid Earnings Engine (Max 140 base or Surge)"]
    PayoutEngine --> EarningsLedger["Earnings Screen & Payout Ledger"]
  
```

```

[Rider Login] ---> [Duty Status Toggle]
                    |
  
```



Detailed Rider App Working Flow

1. Continuous Background GPS Engine:

- Operates a persistent Android Foreground Service isolate (`RiderBackgroundTaskHandler`), ensuring GPS coordinates stream to the server even when the phone screen is locked.
- 30-Second Countdown Dispatch:
 - When a store marks an order ready, all eligible free riders receive an interactive popup with delivery distance, earnings preview, and a 30-second countdown.
 - When accepted, the backend cancels the alert on all other riders' screens (`order:dispatch_cancelled`).
- Hybrid Earnings Model:
 - Guarantees ** flat minimum base pay** per delivery, or the full customer delivery surcharge .

2.4 Admin Web Portal Flowchart

```

flowchart LR
    AdminLogin["AdminLogin[\"Municipal Admin Login\"]"] --> AdminOverview["AdminOverview[\"Admin Control Dashboard\"]"]
    AdminOverview --> MerchantKYC["MerchantKYC[\"Merchant Verification Queue\"]"]
    AdminOverview --> GrievanceDesk["GrievanceDesk[\"Civic Grievance Resolution\"]"]
    AdminOverview --> LiveRiders["LiveRiders[\"Live Fleet GPS Radar Map\"]"]
    AdminOverview --> Financials["Financials[\"Financial Ledger & Settlements\"]"]

    MerchantKYC --> ApproveLicense["|\"Approve License\"|"]
    ApproveLicense --> ApproveMerchant["ApproveMerchant[\"Issue Official Store Badge\"]"]
    MerchantKYC --> RejectLicense["|\"Reject License\"|"]
    RejectLicense --> RejectMerchant["RejectMerchant[\"Record Reason & Notify\"]"]

```

ADMIN MUNICIPAL PORTAL	
Merchant KYC Queue - Verify License Doc - Verify Bank Details - Approve / Reject Store	Civic Grievance Desk - Triage Citizen Issues - Dispatch Field Officer - Award Resolution Coins (500)
Live Delivery Radar Map - Live Rider GPS Tracking - Order Status Monitoring	Financial Ledgers & Audits - Store Settlements - Rider Hybrid Payout Ledger

3. Complete Backend API Directory

Base URL: `http://<HOST>:5000/api`

3.1 Authentication & Profile (/api/auth)

Method	Endpoint	Description	Request Payload / Params
POST	/send-otp	Sends SMS OTP to citizen/rider/store phone	{ phone, role }
POST	/verify-otp	Validates OTP and returns auth session	{ phone, otp, role }
POST	/profile	Creates or updates user profile metadata	{ userId, name, email, address, role }
GET	/profile/:userId	Retrieves user profile by ID or phone number	userId (Path parameter)
POST	/update-fcm-token	Registers Firebase Cloud Messaging push token	{ userId, fcmToken }
POST	/send-phone-update-otp	Sends OTP verification to new phone number	{ oldPhone, newPhone }
POST	/verify-phone-update-otp	Confirms phone migration across all databases	{ oldPhone, newPhone, otp }
POST	/logout	Invalidates push session and terminates active sockets	{ userId }

3.2 Store Management (/api/stores)

Method	Endpoint	Description	Request Payload / Params
POST	/register	Submits merchant KYC application	{ storeId, name, phone, address, timings, bankDetails, licenseNumber, storeImage }
GET	/approved	Fetches all active approved government stores	Query parameters: category, pincode
GET	/pending	Admin queue of unapproved merchant licenses	Header: Admin-Authorization
GET	/my-store/:identifier	Fetches store details by owner ID or phone	identifier (Path parameter)
GET	/:storeId	Public store details and contact info	storeId (Path parameter)
PATCH	/:storeId/online	Toggles store availability for customer orders	{ isOnline: true/false }
PATCH	/:storeId/status	Municipal admin approves or rejects merchant license	{ status: "approved" / "rejected", verifiedBy, reason }
PATCH	/:storeId/profile	Updates store timings, address, and bank details	Updated fields

3.3 Orders & Real-Time Dispatch (/api/orders)

Method	Endpoint	Description	Request Payload / Params
POST	/	Creates a new order and triggers store socket alert	{ userId, storeId, items, itemTotal, deliveryCharge, grandTotal, paymentMethod, deliveryAddress }
GET	/user/:userId	Retrieves order history for a citizen	userId (Path parameter)
GET	/store/:storeId	Retrieves active queue and past orders for a store	storeId (Path parameter)
GET	/available	Retrieves unassigned orders ready for rider dispatch	Query: lat, lng
GET	/rider/:riderId	Retrieves active deliveries assigned to a rider	riderId (Path parameter)
GET	/:orderId	Live detailed order status, rider GPS, and items	orderId (Path parameter)
POST	/:orderId/accept-rider	Locks dispatch to rider & terminates other alerts	{ riderId, riderName, phone }
PATCH	/:orderId/status	Transitions order state (preparing, delivered, cancelled)	{ status: "preparing" "ready_for_pickup" "out_for_delivery" "delivered" "cancelled" }
DELETE	/:orderId	Deletes or archives order from database	orderId (Path parameter)
GET	/riders/live-locations	Admin live GPS beacon map of all active riders	Header: Admin-Authorization

3.4 Products & Inventory (/api/products)

Method	Endpoint	Description	Request Payload / Params
POST	/	Store adds product with price, stock, and unit	{ storeId, title, price, originalPrice, stock, category, unit, image }
GET	/store/:storeId	Retrieves product catalog for a specific store	storeId (Path parameter)
GET	/:productId	Retrieves single product details	productId (Path parameter)
PATCH	/:productId	Updates price, stock levels, or images	{ price, stock, isAvailable }
DELETE	/:productId	Removes product from catalog	productId (Path parameter)

3.5 Citizen Wallet & Refunds (/api/wallet)

Method	Endpoint	Description	Request Payload / Params
GET	/:userId	Retrieves balance, coins, and ledger transactions	userId (Path parameter)
POST	/topup	Adds funds to wallet balance via UPI / Card	{ userId, amount, paymentMethod, referenceId }
POST	/deduct	Atomic debit during order placement	{ userId, amount, orderId, title, subtitle }
POST	/refund	Direct refund credit for cancelled orders	{ userId, amount, orderId, reason }
POST	/redeem-coins	Converts civic complaint coins to wallet cash	{ userId, coins } (100 coins = ₹1)

3.6 Civic Grievances & Complaints (/api/complaints)

Method	Endpoint	Description	Request Payload / Params
POST	/	Citizens submit grievance with geotag & photo	{ userId, title, description, category, address, location: { lat, lng }, image }
GET	/	Public civic feed of community issues	Query: category, status, page
GET	/user/:userId	List of grievances posted by current citizen	userId (Path parameter)
GET	/meta/stats	Municipal stats: Total, In Progress, Resolved	Public summary
POST	/:complaintId/like	Upvote civic grievance	{ userId }
POST	/:complaintId/comment	Add community update or feedback	{ userId, userName, comment }
PATCH	/:complaintId/status	Municipal Officer resolves grievance & awards coins	{ status: "Resolved", resolutionProof, coinsAwarded: 500 }

3.7 Delivery Addresses (/api/address)

Method	Endpoint	Description	Request Payload / Params
GET	/:userId	Retrieves saved delivery addresses	userId (Path parameter)
POST	/	Saves home / work / other delivery address	{ userId, tag, address, landmark, pincode, lat, lng, isDefault }
PUT	/:id	Modifies existing address record	Address fields
DELETE	/:id	Deletes delivery address	id (Path parameter)

3.8 Media File Uploads (/api/upload)

Method	Endpoint	Description	Form-Data
POST	/store	Uploads merchant license or storefront image	image (Multipart/form-data)
POST	/product	Uploads grocery/medicine product photo	image (Multipart/form-data)
POST	/profile	Uploads citizen/rider avatar photo	image (Multipart/form-data)
POST	/	Uploads grievance photo with auto EXIF strip	image (Multipart/form-data)

4. Real-Time WebSocket Architecture (Socket.io)

Event Matrix

Event Name	Sender	Listener Room	Payload	Purpose
order:new	Backend	store:<storeId>	Order Object	Alerts merchant of incoming order with audio cue
order:status_update	Store / Rider	order:<orderId>, store:<storeId>	{ orderId, status }	Real-time stepper progression for customer
order:dispatch_alert	Backend	riders	Order Summary + 30s countdown	Broadcasts delivery opportunity to all free riders
order:dispatch_cancelled	Backend	riders	{ orderId }	Immediately closes dispatch popup on all other riders
rider:location	Rider FG Service	order:<orderId>	{ lat, lng, bearing, orderId }	Smooth vehicle marker movement on citizen map
chat:send	Citizen / Rider	chat:<orderId>	{ senderId, message, timestamp }	In-app delivery coordination chat
complaint_created	Citizen	public:complaints	Grievance Object	Live municipal grievance board updates